



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (`__init__`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

P – Performance Measure - Evaluates the success of the agent.
E – Environment - The world in which the agent operates.
A – Actuators - The mechanisms used by the agent to perform actions.
S – Sensors - The mechanisms used by the agent to collect information from the environment.

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

`self.width` – Width of the grid.
`self.height` – Height of the grid.
`self.walls` – Positions of the walls.
`self.food_positions` – Positions of all food items.
`self.opponents` – Positions of the opponent agents.
`self.agent_pos` – Current position of the main agent within the environment.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi-Agent

The baseline environment is Multi-Agent because the code includes `self.opponents`, which creates opponent agents that move in the environment. Therefore, the main agent shares the environment with other agents that can influence its performance.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of all the percepts (information) that an agent has received from the time it started until the current moment. The agent uses this history to decide what action to take next.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

The `get_percept()` method represents the Sensors (S) component of the PEAS framework. Because it gathers information about the environment and provides it to the agent. It returns the agent's position, opponent positions, food status, wall status, collision status, score, and remaining food.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

The environment is Fully Observable because the `get_percept()` method gives the agent all the important information about the current environment, such as its position, opponent positions, food status, collision status, score and remaining food. Therefore, the agent can observe all the relevant information needed to make

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

The Performance Measure is used to evaluate how well the agent is performing. It increases or decreases based on the agent's actions.

When `execute_action()` deducts points for hitting a wall, it updates the Performance Measure (P) because the agent's score is reduced. The score is used to measure 

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

It is structurally important because the performance measure should evaluate the actual results of the agent's actions in the environment, not its internal processing.

Deducting points for hitting a wall reflects a poor action that affects the environment, whereas measuring internal processing could encourage inefficient behaviour instead of achieving the task goal. 

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

- Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

If `self.toxic_traps` is added to the environment but hidden from the agent's sensors, the environment becomes Partially Observable.

This is because the agent cannot perceive the locations of the toxic traps, so it does not have complete information about the environment when making decisions. 

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Adding `'smells_toxin'` expands the agent's percept sequence by providing information about nearby toxic traps.

This helps the agent make better decisions, avoid dangerous locations and maximize its expected utility. 

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The Vacuum World Exploit was a robot vacuum that earned points every time it cleaned dirt.

Instead of keeping the room clean, it repeatedly cleaned the same dirt by dumping it back onto the floor and cleaning it again to gain unlimited points.

This showed why performance measures should evaluate the actual state of the 

Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING